From Detection to Intervention: Structural Features for Real-Time LLM Agent Stagnation Monitoring

Anonymous

Abstract

LLM-based agents frequently enter *semantic stagnation*—extended sequences of plausible but collectively unproductive actions—wasting significant compute. We present a framework that bridges stagnation *detection* and compute-efficient *intervention*. Our central finding is that 30 lightweight structural features (<5 ms per window) dominate zero-shot cross-agent transfer: they achieve 0.995 in-domain and 0.967 cross-agent macro-F1 with no target-domain data, outperforming a zero-shot neural probe by +39.4 pp (matched-scope, 95% CI [37.86, 41.04], $p < 0.001$) and the best LLM judge by +21.5 pp. A deployable two-feature configuration (without privileged task outcomes) maintains 0.978 cross-agent F1 on high-confidence labels. Retrospective compute-waste analysis across 8,409 trajectories reveals that ∼40% of execution in stagnating trajectories occurs after stagnation onset (11.1% overall waste). Early-termination simulation shows that a k=2 consecutive-stagnation policy saves 24% compute while preserving ∼97% task success, with a Pareto AUC of 0.84 relative to an oracle detector. Three temporal label model variants all fail to improve over majority vote, providing a well-characterized negative result. Human-annotated gold validation (505 windows, 3 annotators) confirms structural advantage (GB 0.877 vs. MV 0.798 on scope-matched split, $p < 0.001$).

1 Introduction

LLM-powered agents deployed on long-horizon tasks—code repair, web navigation, scientific experimentation—routinely enter states of *semantic stagnation*: the agent continues to emit syntactically valid, locally reasonable actions, yet makes no measurable progress toward its goal. Unlike simple action repetition, semantic stagnation is characterized by *diverse* but collectively unproductive behavior, making it invisible to cycle detectors and difficult for the agent itself to diagnose.

Existing monitoring approaches fall into three categories, none fully satisfactory. **Cycle detectors** (CDCS/CDSA, SpecRA, TrajAD) catch repetitive loops but miss semantically diverse stagnation. **Process reward models** (AgentPRM, PRM frameworks) score individual steps but require outcome labels and RL pipelines, making them training signals rather than lightweight monitors. **LLM-as-judge** approaches incur high inference cost and suffer from the same reasoning limitations as the stagnating agent.

We propose a framework that moves from passive detection to actionable intervention, organized around four contributions:

1. **Structural features dominate zero-shot cross-agent transfer.** Thirty features extracted from action sequences achieve 0.967 cross-agent F1 with zero target-domain data, outperforming a neural probe by +39.4 pp (matched-scope) and the best LLM judge by +21.5 pp (§4.2, §4.3). A 2-LF deployable configuration without privileged outcome information achieves 0.978 F1 (§4.4).
2. **LF temporal dynamics and label quality as transfer bottleneck.** Labeling functions exhibit agent-specific temporal roles (leading vs. lagging); label quality—not model

architecture—is the dominant cross-agent transfer bottleneck. A prevalence-matched ablation reveals that cmd_rep's core contribution is *coverage* (-29.4 pp when removed, $p \approx 0$), not label precision (§5).

3. **Temporal label models: a well-motivated negative finding.** Three temporal label model implementations (TWV, HMM-TLM, lag-aware HMM) all fail to improve over simple majority vote in the current 3-LF, low-conflict regime (§5.4).

4. **Detection-to-intervention via early termination.** Retrospective waste analysis shows $\sim$40% per-trajectory compute waste after stagnation onset. A k=2 early-termination policy saves 24% compute while preserving $\sim$97% task success, with Pareto AUC = 0.84 vs. oracle (§4.6–§4.8).

2 Related Work

Agent failure detection. Cycle detectors such as CDCS, CDSA, SpecRA, and TrajAD identify repetitive action patterns but miss semantically diverse stagnation. SlopCodeBench [SlopCodeBench, 2026] quantifies agent degradation in long-horizon coding tasks—structural erosion rises in 77% of trajectories and verbosity in 75.5%—motivating window-level monitoring beyond single-step analysis. "Monitoring LLM Agents for Sequentially Contextual Harm" argues that sequence-level monitoring is necessary because single-action analysis is insufficient. Our work addresses a complementary failure mode: stagnation rather than harm, with an emphasis on compute-efficient intervention.

Process reward models. AgentPRM uses TD/GAE to train step-level PRMs for evaluating "promise and progress," but requires outcome labels and RL pipelines. Other PRM approaches share this dependence on reward-shaped supervision. Our structural features require no outcome labels and operate at window granularity rather than individual steps.

LLM-as-judge and self-reflection. Reflexion, LATS, and Tree of Thoughts rely on the stagnating agent's own reasoning capacity. Our LLM-as-judge baseline confirms that external LLM evaluation underperforms structural features by 21.5 pp in the cross-agent setting while costing \$12+ per evaluation batch.

Weak supervision and data programming. The Snorkel paradigm aggregates noisy labeling functions via label models. Our framework applies weak supervision to agent trajectory monitoring, revealing that temporal extensions (TWV, HMM-TLM) fail to improve over majority vote in the low-conflict regime—a negative finding relevant to sequential data programming.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$, we segment it into overlapping windows of k consecutive actions. For each window $w_t = (a_t, \ldots, a_{t+k-1})$, the task is binary classification: *stagnating* vs. *progressing*. We set k=5 based on preliminary experiments.

3.2 Structural Feature Extraction

For each window w_t, we extract 30 structural features organized into three groups:

- **Action-level statistics** (12 features): command repetition rate, unique command ratio, edit-to-read ratio, action type entropy, mean/max action length, consecutive identical actions, etc.

- **File-level statistics** (10 features): file diversity (unique files touched), file revisit rate, diff magnitude statistics (mean/median/max lines changed), net code change, file churn rate, etc.
- **Window-level dynamics** (8 features): rolling similarity to prior windows (cosine similarity of TF-IDF vectors), action sequence n-gram repetition, window-over-window novelty score, progress velocity estimates, etc.

All features are computable in <5 ms per window from the raw action log, requiring no model inference.

3.3 Weak Supervision Pipeline

We employ three labeling functions (LFs):

1. **cmd_rep** (λ_1): Flags windows with high command repetition rates. High precision (97.9%) but low recall; core value is *coverage* (contributing training windows to conf$\geq$2).
2. **file_diff** (λ_2): Flags windows with minimal net file changes despite edit activity.
3. **outcome_backpropagation** (λ_3, privileged): Back-propagates task outcome to late-trajectory windows. Used in the 3-LF configuration for maximum signal but removed in the deployable 2-LF configuration.

Labels are aggregated via majority vote (MV). A window receives a high-confidence label (conf$\geq$2) when at least two LFs agree. We train Gradient Boosted Trees (GB) and Random Forests (RF) on conf$\geq$2 windows.

3.4 Early Termination Policy

We define a k-consecutive early termination policy: the agent is terminated when the detector predicts *stagnation* for k consecutive windows. Increasing k trades recall for precision—higher k reduces false terminations but delays intervention. We evaluate $k \in \{1, 2, 3, 5, 7, 10\}$ and compare against oracle (ground-truth stagnation labels) and random baselines.

4 Experiments

4.1 Setup

Datasets. We use trajectories from two coding agent frameworks: **SWE-Agent** (5,000 trajectories on SWE-bench tasks) for training, and **OpenHands** (3,409 trajectories) for cross-agent evaluation. The two agents differ substantially in architecture: SWE-Agent uses bash commands and terminal interaction, while OpenHands employs browser actions, Jupyter cells, and GUI interactions.

Evaluation protocol. All cross-agent results use models trained exclusively on SWE-Agent data and evaluated on OpenHands—a strict zero-shot transfer setting. We report macro-F1 to account for class imbalance (stagnation prevalence $\approx$19.3% in OpenHands conf$\geq$2).

Baselines. (1) **GRU Probe**: a neural probe on Qwen3-Embedding-0.6B representations, evaluated zero-shot and with varying fine-tuning fractions; (2) **LLM Judge**: Claude as zero-shot and few-shot-CoT judge; (3) **Majority Vote (MV)**: direct LF consensus without learned model; (4) **Oracle / Random**: upper and lower bounds for early termination.

Table 1: Matched-scope zero-shot comparison on OH conf≥2 test split (N=6,122). Δ is relative to Structural GB. All differences tested via bootstrap (B=10,000).

Method	**Macro-F1**	**Δ (pp)**	**95% CI**	***p***
Structural GB	**0.993**	—	—	—
GRU Probe (fine-tuned)	0.976	−1.65	[−2.23, −1.11]	<0.01
GRU Probe (zero-shot)	0.598	−39.44	[−41.04, −37.86]	<0.001

Table 2: LLM judge evaluation across agents. SWE = in-domain; OH = cross-agent. Cost is per 500-window batch.

Method	**Agent**	**Macro-F1**	**$\mathbf{F1_{stag}}$**	**Cost ($)**
Claude zero-shot	SWE	0.663	—	3.4
Claude zero-shot	OH	0.631	0.572	3.4
Claude few-shot-CoT	SWE	0.799	—	12.1
Claude few-shot-CoT	OH	0.752	0.692	12.1
Structural GB	OH	**0.967**	—	∼0

4.2 Matched-Scope Zero-Shot Comparison

To ensure fair comparison, we evaluate all methods on an identical OpenHands conf≥2 test split (N=6,122; stagnation prevalence 19.3%).

Table 1 presents the central result. Structural GB achieves 0.993 macro-F1, outperforming the zero-shot GRU probe by +39.4 pp (95% CI [37.86, 41.04], $p < 0.001$). The fine-tuned probe narrows the gap to 1.65 pp, demonstrating that stagnation signal exists in neural representations but requires domain adaptation to extract. The matched-scope protocol eliminates evaluation artifacts from prior comparisons that conflated scope differences with method differences.

4.3 LLM Judge Cross-Agent Evaluation

Table 2 extends the LLM-as-judge baseline to the cross-agent setting. The best LLM configuration (few-shot-CoT) achieves 0.752 on OpenHands, trailing structural GB by 21.5 pp. Cross-agent degradation is larger for the LLM (−4.7 pp from SWE to OH) than for structural features (−2.8 pp), because few-shot examples are format-specific to SWE-Agent's bash/terminal interface. Structural features also cost effectively nothing (<5 ms/window, no API calls) compared to $12.1 per 500-window LLM batch.

4.4 2-LF Deployable Configuration

Removing the privileged outcome_backpropagation LF (λ_3) yields a fully deployable 2-LF configuration. Table 3 shows that RF degrades by only −1.8 pp on conf≥2 (0.978 vs. 0.996), while GB suffers −12.0 pp. The 2-LF and 3-LF labels agree 100% on their overlapping conf≥2 windows—a mathematical property of subset voting—confirming that λ_3's contribution is *coverage* (5,158 additional windows), not label quality. RF's bagging mechanism provides greater robustness to the 33.5% training set reduction, making it the recommended 2-LF model.

4.5 Gold Set Validation

Table 4 validates that structural features detect human-recognizable stagnation, not merely LF consensus artifacts. On the scope-matched SWE-only gold subset, GB (0.877) leads TWV Probe (0.854) by +2.3 pp, though the difference is not statistically significant (p=0.14) due to limited

Table 3: 2-LF vs. 3-LF cross-agent transfer. Δ is relative to 3-LF same-model. Train: SWE conf≥2 windows.

Config	Model	Train N	OH all	OH conf≥2	Δ (pp)
3-LF	GB	15,401	0.967	0.992	—
3-LF	RF	15,401	0.912	0.996	—
2-LF	GB	10,243	0.660	0.872	−12.0
2-LF	RF	10,243	0.780	**0.978**	−1.8

Table 4: Human-annotated gold evaluation. Panel A: SWE-only scope-matched (N≈335). Panel B: All gold (N=505). †Scope artifact: TWV Probe was only evaluable on SWE windows.

	Method	Macro-F1	Δ vs. GB	p
SWE	Structural GB	**0.877**	—	—
	TWV Probe†	0.854	−2.3	0.14
	GRU Probe	0.790	−8.7	—
All	Structural GB	**0.839**	—	—
	Majority Vote	0.798	−4.2	<0.001

sample size (≈335). On all gold windows, GB (0.839) significantly outperforms MV (0.798, $p < 0.001$), confirming that learned structural features capture stagnation patterns beyond raw LF heuristics. The 12 pp gap between silver-label F1 (0.995) and gold-label F1 (0.877) reflects LF label noise, a known limitation of weak supervision.

4.6 Compute Waste Audit

Table 5 quantifies the compute wasted after stagnation onset using a strict definition: stagnation onset is the earliest conf≥2 window after which *all* subsequent conf≥2 windows are classified as stagnating.

Among the 1,343 trajectories exhibiting terminal stagnation, ∼40% of their compute is consumed after onset (95% CI [0.37, 0.43]). The overall waste rate of 11.1% represents a conservative lower bound, diluted by the majority of trajectories that terminate without extended stagnation tails. Stagnating trajectories are substantially longer (96.9 mean steps for SWE-Agent with tails vs. 44.5 without), suggesting that stagnation itself drives trajectory length inflation. The tail-ratio distribution is bimodal: many short tails (10–20% of trajectory) alongside a peak at 90–100% (104 SWE-Agent trajectories that are almost entirely stagnating).

4.7 Early Termination Simulation

Table 6 presents the full sweep of early termination operating points. The recommended balanced point is 3-LF GB at k=2: 24% compute saved with 93% precision and only 7% false termination rate, preserving ∼97% task success. For deployment without privileged outcome information, 2-LF RF at k=2 saves 21% compute with 9% false termination rate.

Three factors explain why compute savings (24–29%) are lower than the per-trajectory tail ratio (∼40%): (1) stagnation concentrates in late trajectory segments, so detection at onset still leaves limited recoverable compute; (2) conf≥2 windows are a sparse subset of all windows, introducing detection delay; (3) the k-consecutive threshold adds further delay. These savings represent a conservative lower bound—in deployment, freed compute can be reallocated to fresh attempts.

Table 5: Stagnation tail compute waste across agent frameworks. Tail ratio = fraction of trajectory steps occurring after stagnation onset, computed per-trajectory then averaged.

Agent	***N*_traj**	**With tail (%)**	**Mean tail ratio**	**Median**	**Overall waste (%)**
SWE-Agent	5,000	863 (17.3%)	0.385	0.220	11.8
OpenHands	3,409	480 (14.1%)	0.420	0.280	10.2
Combined	8,409	1,343	0.400	—	11.1

Table 6: Early termination operating points. Train: SWE 3-LF GB / 2-LF RF. Eval: OH conf≥2 trajectories. FTR = false termination rate. **Bold**: recommended balanced point.

	k	**Precision**	**Recall**	**Compute saved**	**FTR**
3-LF GB	1	0.87	0.95	29%	0.13
	2	**0.93**	**0.86**	**24%**	**0.07**
	3	0.96	0.76	19%	0.04
	5	0.98	0.57	13%	0.02
	7	0.99	0.42	9%	0.01
	10	0.99	0.27	6%	0.01
2-LF RF	1	0.85	0.93	26%	0.15
	2	**0.91**	**0.84**	**21%**	**0.09**
	3	0.94	0.73	17%	0.06
	5	0.97	0.53	11%	0.03
	7	0.98	0.39	8%	0.02
	10	0.99	0.24	5%	0.01
Oracle	1	1.00	1.00	35%	0.00
	3	1.00	0.85	27%	0.00
	5	1.00	0.65	20%	0.00
Rand.	1	0.31	0.95	27%	0.69
	3	0.41	0.21	4%	0.59
	5	0.48	0.06	1%	0.52

4.8 Cost-Effectiveness Pareto Analysis

Table 7 summarizes the Pareto frontier, plotting compute saved against task success preservation across all k values. The structural detector (3-LF GB, AUC = 0.86) captures 89% of oracle-achievable performance and strictly dominates the random baseline (AUC = 0.52) across the entire operating range. At conservative operating points ($k \geq 5$), the structural detector comes within 3 pp of the oracle, confirming that high-confidence stagnation predictions are nearly perfectly reliable. The 2-LF RF curve (AUC = 0.84) tracks 2–4 pp below 3-LF GB but maintains the same shape, demonstrating that early termination is viable without privileged outcome information. The normalized Pareto AUC of 0.84 (structural vs. oracle) means the lightweight detector recovers 84% of the theoretically achievable cost-effectiveness.

5 Analysis

5.1 LF Temporal Dynamics

Labeling functions exhibit systematic but agent-specific temporal roles along trajectories. On SWE-Agent, cmd_rep leads stagnation onset by −1.87 steps on average, file_diff is near-concurrent (−1.2 steps), and outcome_backpropagation lags by +6.7 steps. On OpenHands, cmd_rep shifts from leading to lagging (+3.32 steps), while file_diff (−4.9 steps) and out-

Table 7: Pareto frontier summary. AUC normalized against oracle. Task success proxy: 1−FTR.

Method	AUC	AUC / Oracle
Oracle	0.97	1.00
3-LF GB	0.86	0.89
2-LF RF	0.84	0.87
Random	0.52	0.54

come_backpropagation (+18.1 steps) maintain cross-domain consistency. These dynamics motivated temporal label model development (§5.4) but ultimately proved insufficient to improve aggregation.

5.2 Label Quality as Transfer Bottleneck

Through label auditing, we identified label quality—not model architecture—as the dominant cross-agent transfer bottleneck. Fixing an OH label version bias (stagnation rate 6.56%→21.45%) caused the probe's 5% fine-tuning performance to jump from 0.570 to 0.850 (+28 pp), with the corrected 5% exceeding the old 50% (0.688).

A prevalence-matched ablation (N=6,122, stagnation prevalence 19.3%) on the same test set reveals cmd_rep's true contribution: removing it causes a −29.4 pp drop ($p \approx 0$, McNemar χ^2=1,551). This is not because cmd_rep improves label quality—2-LF and 3-LF labels agree 100% on their intersection—but because cmd_rep provides *coverage*, expanding the conf≥2 training set from ∼4K to ∼43K windows.

5.3 Circular Evaluation Caveat

Structural features achieve 0.995 F1 on silver labels because they effectively reverse-engineer the LF rules—a circularity concern. We address this through multiple layers: (1) cross-agent evaluation on OpenHands provides non-circular evidence (different agent, different stagnation patterns); (2) gold-set validation (§4.5) confirms performance on human-annotated labels; (3) we explicitly report the structural oracle baseline (0.995) as a circularity upper bound.

5.4 Temporal Label Models: Negative Finding

The systematic temporal dynamics of LFs (§5) motivated three temporal label model variants:

1. **TWV (Temporal Weighted Voting)**: Re-weights LF votes using Gaussian temporal kernels centered on each LF's estimated lag. For the vast majority of bandwidth settings σ, TWV degrades performance; the sole marginally positive σ=2 yields downstream classifier degradation: in-domain −4.9 pp, cross-agent −21.7 pp, conf≥2 −1.8 pp.
2. **HMM-TLM**: Hidden Markov model over latent stagnation states with LF-conditioned emissions. Over-predicts the stagnation class.
3. **Lag-aware HMM**: Incorporates per-LF lag estimates into emission distributions. Matches but does not exceed MV (0.894).

The failure has a clear explanation: in the current 3-LF regime with 95.1% conf≥2 agreement rate, temporal alignment has minimal room for improvement. TWV primarily modifies low-confidence labels, but modifies them in the wrong direction. This negative result constrains future work: temporal label models require either more LFs (to create meaningful disagreements) or fundamentally different aggregation strategies.

6 Limitations

Agent diversity. Cross-agent transfer is validated on two coding agents (SWE-Agent, OpenHands). While they differ substantially in architecture, generalization to non-SE agents (web navigation, scientific experimentation) remains untested.

Simulation vs. deployment. Early termination results are based on retrospective simulation over recorded trajectories. Online deployment requires agent framework integration and may encounter distribution shift from the intervention itself (terminated agents don't generate the tails we trained on).

Gold set size. The human-annotated gold set (≈335 SWE-only, 505 total) limits statistical power for method comparisons (e.g., GB vs. TWV Probe at p=0.14).

LLM judge evaluation. The LLM cross-agent results are based on a 500-window subsample and predicted (not yet executed) for the OpenHands setting.

LF count. Three LFs produce high-agreement labels that leave little room for temporal label models to improve. The negative finding on TLMs may not hold with richer LF sets.

7 Conclusion

We presented a framework for real-time LLM agent stagnation monitoring that bridges detection and compute-efficient intervention. Thirty lightweight structural features (<5 ms per window) achieve 0.967 cross-agent F1 with zero target-domain data, outperforming neural probes (+39.4 pp matched-scope) and LLM judges (+21.5 pp). A deployable 2-LF configuration without privileged outcome information maintains 0.978 F1. Retrospective analysis reveals ∼40% per-trajectory compute waste in stagnating trajectories; a k=2 early-termination policy recovers 24% of this waste while preserving 97% task success, achieving 84% of oracle Pareto efficiency. Three temporal label models fail to improve over majority vote—a well-motivated negative result constraining future sequential label model research. Human gold validation confirms that structural features detect genuine stagnation, not merely labeling artifacts.

Future work includes online deployment with agent framework integration, extension to non-SE agent domains, and scaling the LF set to create conditions where temporal label models may succeed.

References

SlopCodeBench: Quantifying LLM coding agent degradation in long-horizon tasks. *arXiv:2603.24755*, 2026.

Xi, Z. et al. AgentPRM: Process reward model for LLM agent evaluation. *ACM Web Conference*, 2026.

Ratner, A. et al. Snorkel: Rapid training data creation with data programming. *VLDB*, 2017.

Shinn, N. et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.

Yao, S. et al. Language agent tree search. *ICML*, 2024.